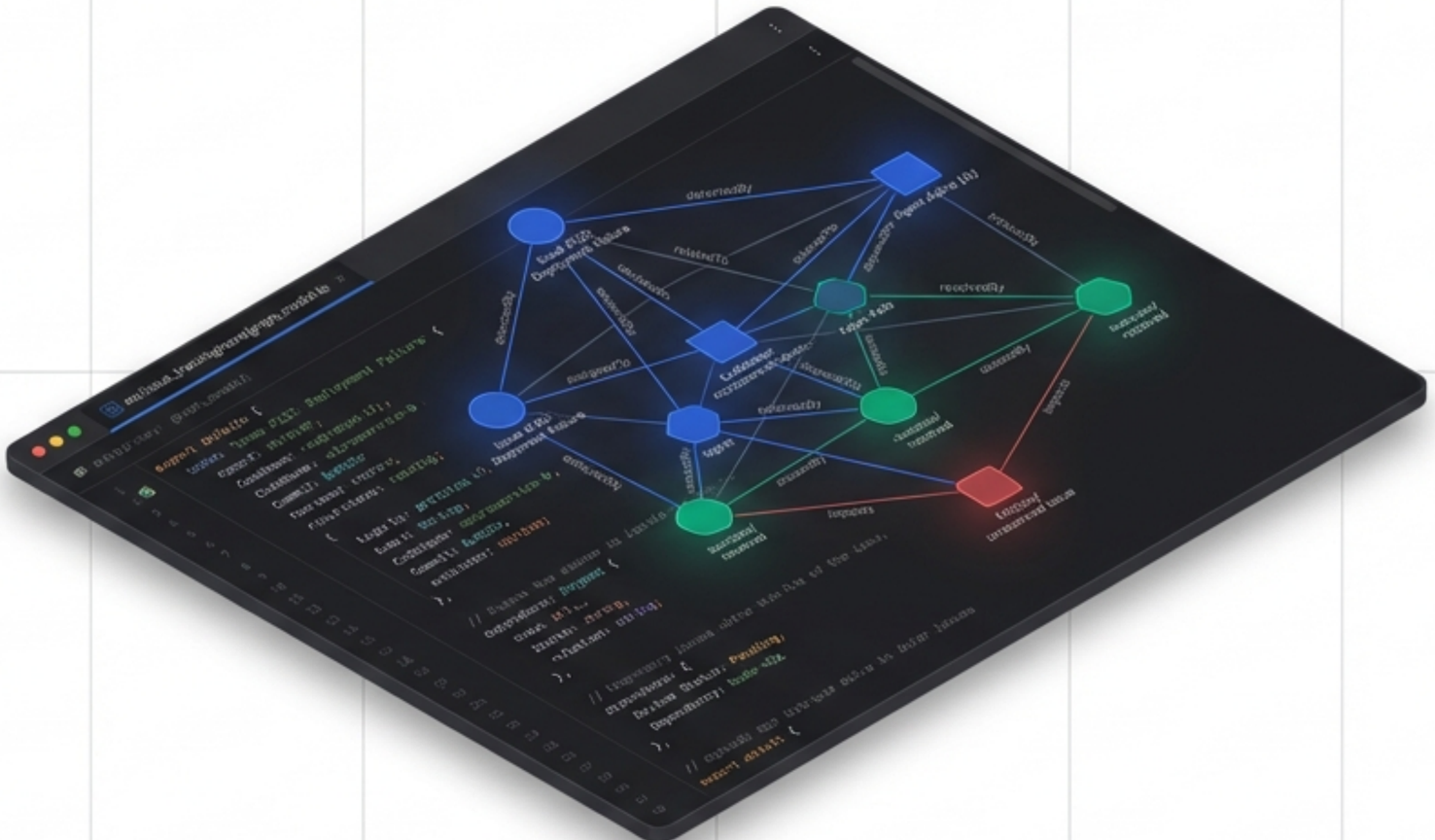
Graph-Based Issue Tracking

Infrastructure for the Age of AI-Agent Collaboration



Technical Executive Briefing | v1.0 | January 31, 2026

Target Audience: CTOs, Architects, AI Leadership

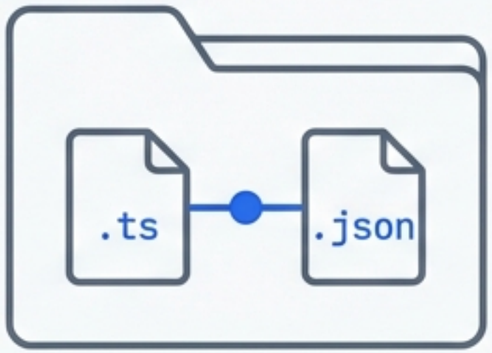
The Repository is the Source of Truth

Moving from SaaS silos to AI-ready infrastructure.

Traditional (Cloud Silo)

- Data trapped in vendor cloud
- Requires authentication flows
- Network connectivity mandatory
- disconnected from code history

Git-Native (Modern)

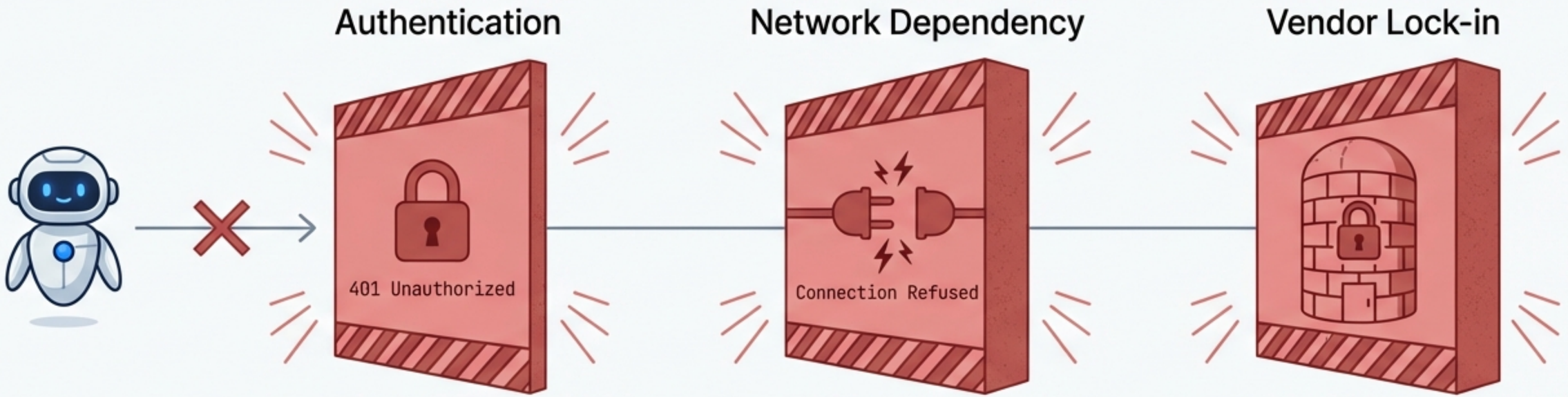


- Data lives in your repository
- Uses native Git credentials
- Works offline / Air-gapped
- Versioned alongside code

> This isn’t just another issue tracker. It’s infrastructure for AI agent collaboration.

The Friction: Why Legacy Tools Fail AI Agents

AI coding assistants like Claude Code and GitHub Copilot are the new workforce, but they hit hard walls when interacting with traditional project management tools (Jira, Linear).



OAuth flows, 2FA, and API tokens create security complexity.

Agents cannot work in offline or air-gapped environments.

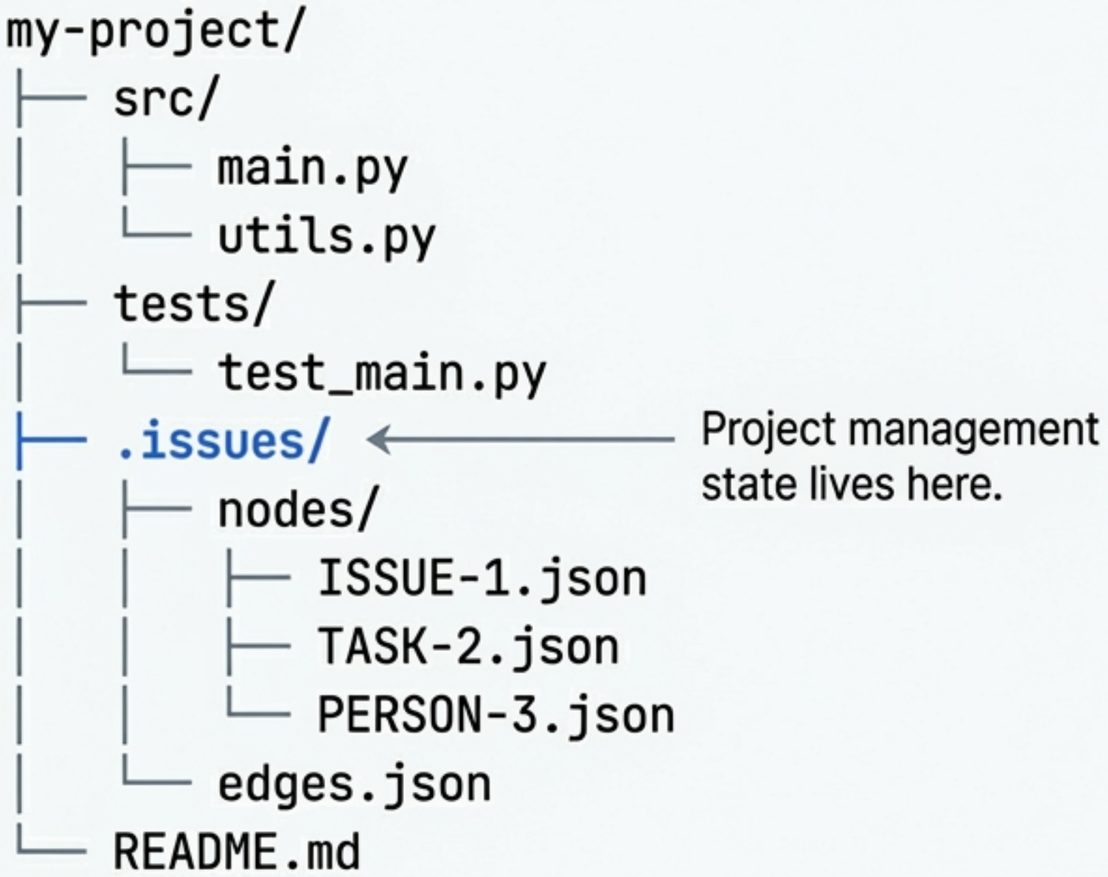
History trapped in proprietary DBs, inaccessible to local context.

Result: High friction, low autonomy, and broken context windows.

The Philosophy: Filesystem as Database

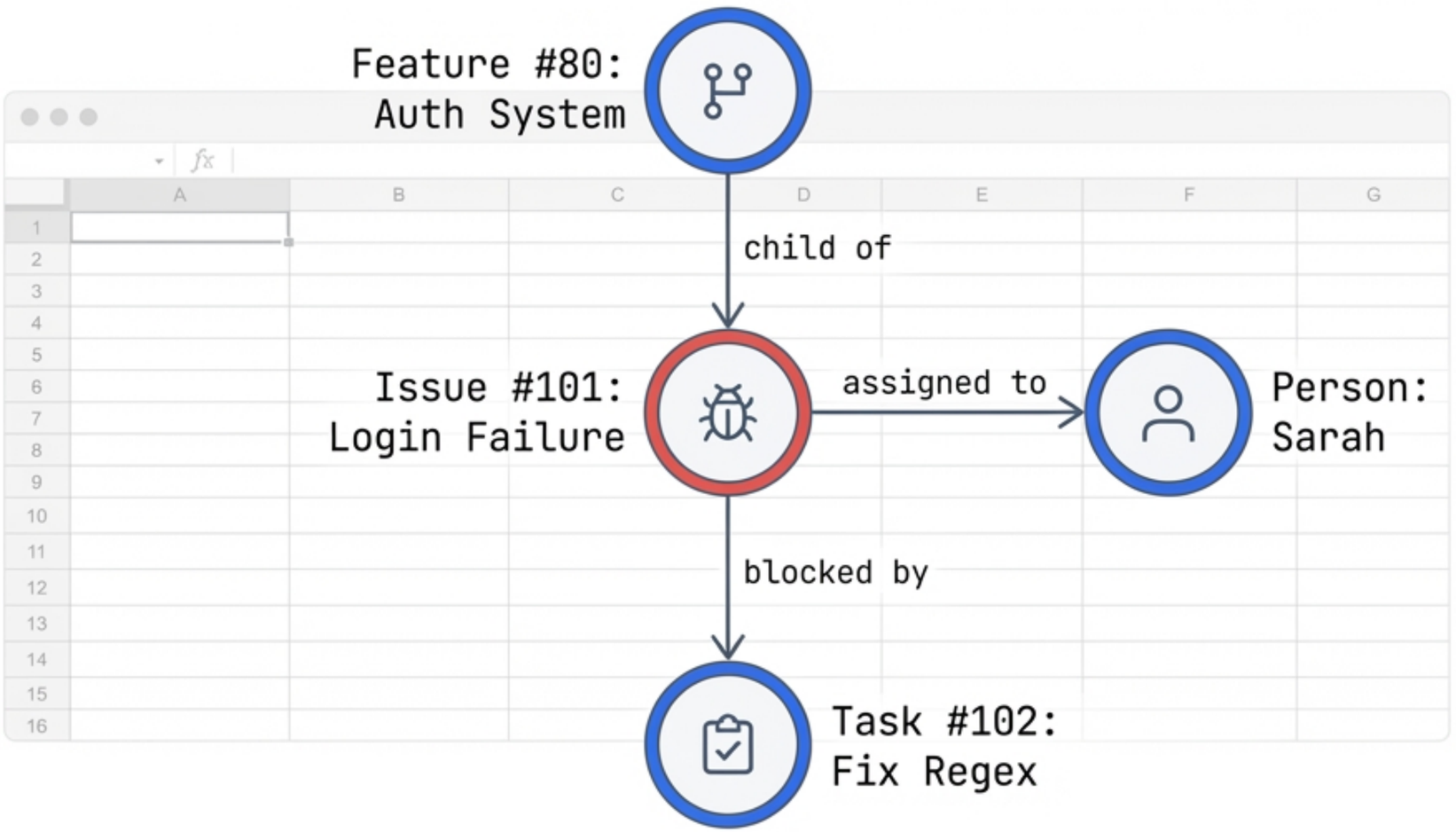
Issues are data, not a service. Extending the Unix philosophy to project management.

- **Core Principle:** If code is text and configuration is text, project management artifacts should be text.
- **Portability:** Issues are backed up, restored, and forked exactly like source code.
- **Versioning:** History is handled natively by Git. No proprietary audit logs required.



The Data Model: From Lists to Graphs

Flat lists (spreadsheets) fail to capture software complexity. We use a graph model where relationships are first-class citizens.



Under the Hood: The Git-Native File Format

Stored in `.issues/` as standard JSON. Human-readable, machine-parsable, and diff-friendly.

Standard JSON Schema

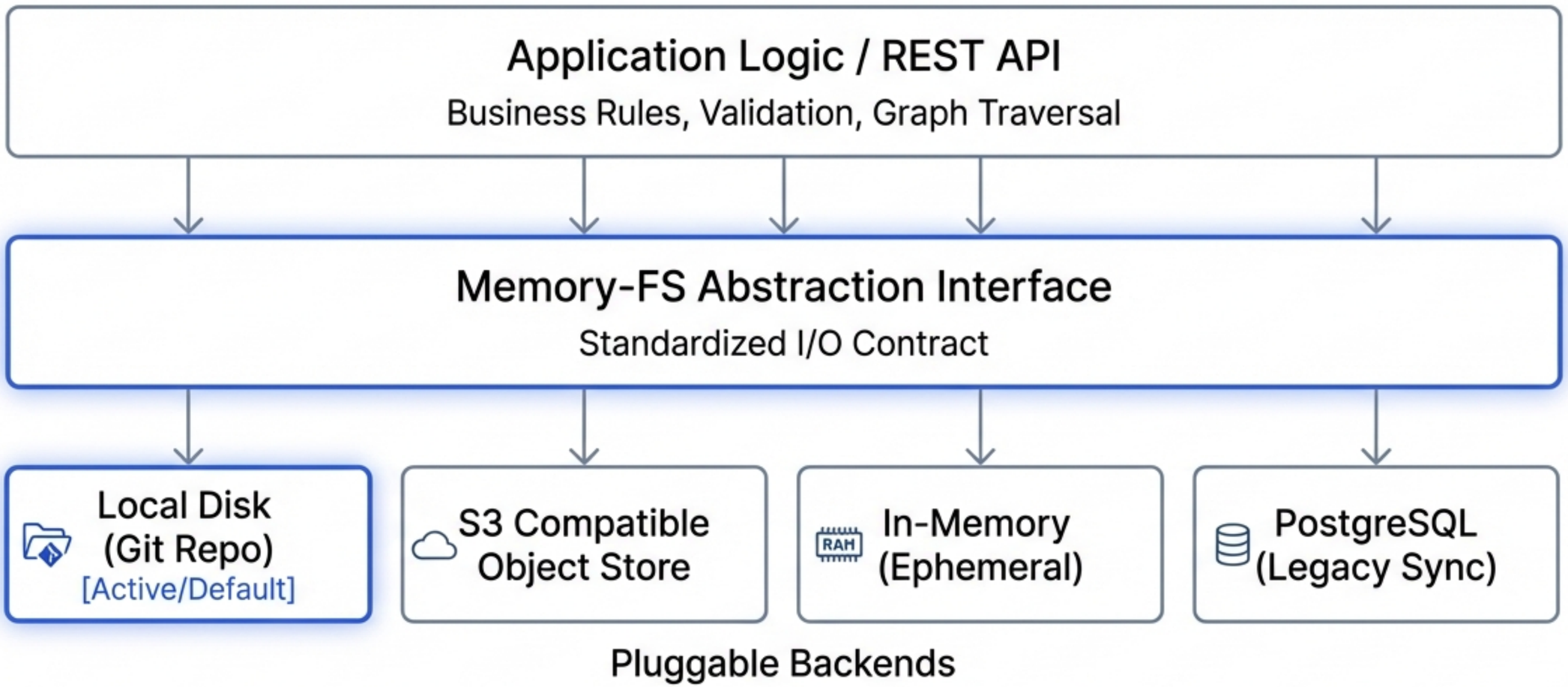
ISSUE-124.json

```
{
   "id": "ISSUE-124",
   "type": "bug",
   "title": "Memory leak in data processor",
   "status": "open",
   "created_by": "user_gh_8821",
   "created_at": "2026-01-30T14:22:00Z",
   "relationships": [
      {
         "target": "FEATURE-12",
         "type": "relates_to"
      },
      {
         "target": "PERSON-04",
         "type": "assigned_to"
      }
   ],
   "body": "Detected increasing heap usage during batch
            processing..."
}
```

Graph Edges defined locally

Architecture: The Memory-FS Abstraction

Decoupling logic from storage for maximum flexibility.



This abstraction allows the system to run in CI/CD (Ephemeral), on a Dev Laptop (Local Disk), or in the Cloud (S3) without code changes.

The Interface: REST API Reference

While data is local, the interface is a standard HTTP API. Agents interact via protocols they already understand—no custom scrapers needed.

API Definition (OpenAPI Style)

`GET /graph/nodes/{id}` - Retrieve node details

`POST /graph/nodes` - Create new issue/task

`GET /graph/traverse?start={id}&depth=3` - Get dependency tree

`PATCH /graph/edges` - Link two nodes

Terminal Output (cURL)

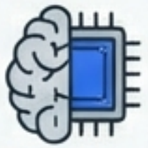
```
$ curl -X POST /graph/nodes \
    -d '{"type":"task", "title":"Refactor Auth"}'

> HTTP/1.1 201 Created
> {
>   "id": "TASK-551",
>   "status": "success",
>   "path": ".issues/nodes/TASK-551.json"
> }
```

Why It Matters: Superior to Legacy SaaS

Feature	Legacy SaaS (Jira/Linear)	Git-Native (Our Solution)
Data Ownership	Vendor Cloud (Locked) ❌	Your Repo (Owned) ✅
Authentication	OAuth / API Tokens ❌	Native Git Credentials ✅
Connectivity	Online Only ❌	Offline / Local First ✅
History Context	Disconnected from Code ❌	Tied to Commit History ✅
Data Format	Proprietary SQL/Blob ❌	Standard JSON ✅

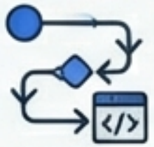
Simplicity enables power. Removing dependencies gains flexibility that traditional architectures cannot match.

Use Cases: Beyond Bug Tracking



LLM Memory & Context

Providing persistent, long-term memory for AI agents. Agents can query the graph to understand project history before writing code.



Execution Flow Logging

Log agent reasoning steps (“thoughts”) as graph nodes linked to the code changes they produced. Full auditability of AI decisions.

DevOps & SRE

Incident tracking lives directly alongside runbooks and IaC. Post-mortems are committed to the repo, not a wiki.

LLMOps

Track model deployment versions, experiments, and evaluation results as structured data within the repository.

Roadmap: From Core to Ecosystem

Phase 1: Foundation (Implemented)

- Core Node/Link CRUD ✅
- Type System (Bug, Task, Feature) ✅
- Memory-FS Abstraction ✅
- REST API v1 ✅
- 4 Storage Backends ✅

Phase 2-5: Advanced Logic (In Progress)

- Cycle Detection Algorithms
- Graph Export (GraphML, DOT)
- Attachment Uploads
- Agent Identity Management

Future Vision: Ecosystem

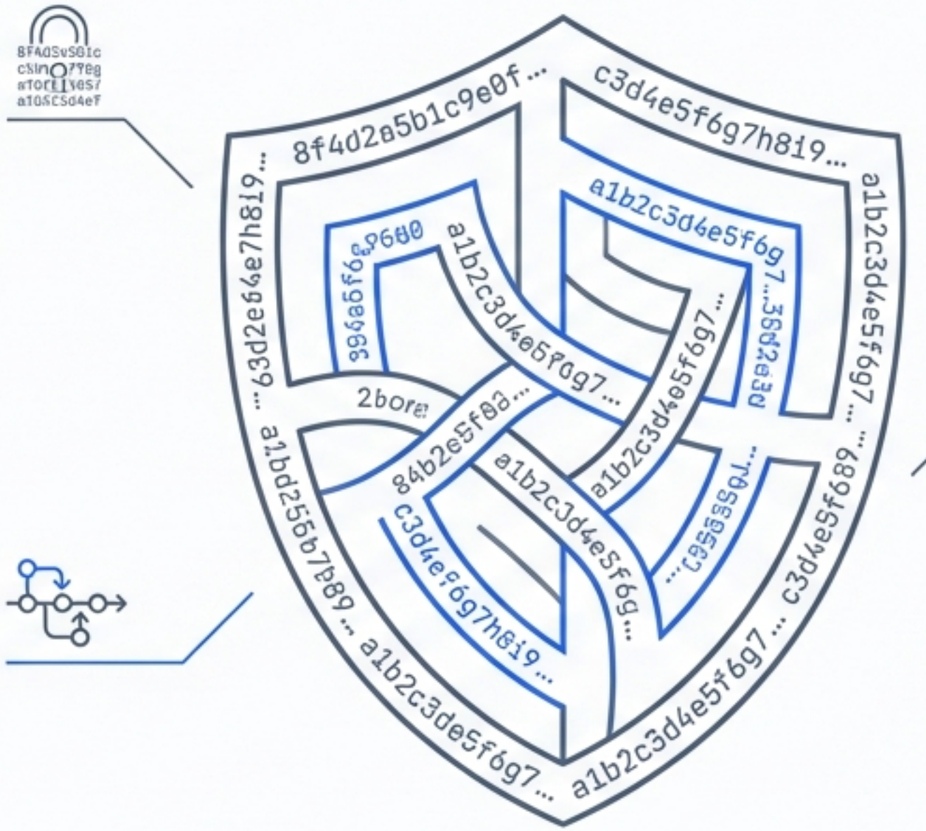
- CLI Tool (`git issue create` in JetBrains Mono)
- VS Code Extension
- AI Triage Bots
- GitHub Action Sync

Security & Governance

Zero-Trust Architecture by Default

Zero-Trust Model

No third-party SaaS credentials to manage. No API tokens to leak. Security relies on existing Git access controls.



Data Sovereignty

Data never leaves your corporate firewall or VPC. It stays exactly where your source code stays.

Immutable Audit Trail

Every change to an issue is a Git commit. Precise attribution of who changed what and when.

The Fundamental Shift

1. Issues are Data.

Store them like any other project artifact. Text-based, versioned, and `local`.

2. AI-First Design.

Designed for machine collaboration from day one, not patched onto legacy UI.

3. Simplicity is Power.

By removing external dependencies, we gain `offline capability`, `branching support`, and `true ownership`.

Data Ownership + AI Readiness

Resources & Contact

Repository:
github.com/your-org/graph-issue-tracker

Documentation:
docs.example.com/issues

API Reference:
api.example.com/docs

Engineering Team:
demo@example.com

View Repository

Built with Type_Safe, Memory-FS, and FastAPI. Designed for humans and AI agents alike.